NETWORK HARDENING PLANNER

Automated Planning project
for AI course

1. Introduction

Enterprise network infrastructures are rarely built from scratch. They accumulate legacy services, deprecated protocols, and open ports over years of organic growth. When a security audit or compliance review demands hardening, network administrators face a combinatorial challenge: which services to disable, which ports to close, which to migrate, and in what order — all without breaking the services that business operations depend on.

This project, Network Hardening Planner, frames network hardening as a formal Automated Planning problem and solves it optimally using the Unified Planning Framework and the Fast Downward solver. Given a machine-readable JSON description of a network (hosts, services, ports, dependencies, vulnerabilities) and a security policy (forbidden ports, forbidden services), the system produces a plan — a legally ordered sequence of hardening actions — that satisfies all security goals without breaking continuity of service.

2. Approach: Classical Planning

The project uses Classical (STRIPS-like + grounding) Planning as its core formalism. This choice is deliberate and well-justified by the structure of the problem.

2.1 Why Classical Planning?

Classical planning operates on a state space defined by a finite set of boolean fluents (propositions) and actions, which have typed preconditions and deterministic effects. Given an initial state, a conjunction of true fluents, and a set of goal conditions, the planner finds a sequence of actions that transitions from initial state to goal state.

Network hardening maps cleanly onto this formalism:

- The state is fully observable: we know which ports are open, which services are active, which are vulnerable.
- Actions are deterministic: blocking a port closes it; patching a service removes its vulnerability.
- Goals are propositional: forbidden ports must be closed; vulnerabilities must be resolved.
- The problem is finite: a bounded number of hosts, ports, and services.

Alternative formalisms — temporal planning, probabilistic planning, hierarchical task networks — add expressive power at the cost of solver availability and computational complexity. For this problem, the additional expressiveness is not needed: risk is modelled as action cost, and timing as plan length.

2.2 The Unified Planning Framework

The Unified Planning (UP) Framework is a Python library that provides a solver-agnostic API for automated planning, allowing problems to be defined once and solved by multiple engines without code changes. This project targets classical, cost-optimal planning and uses Fast Downward as the backend engine. It supports optimal planning via A* search with admissible heuristics (LM-cut, merge-and-shrink), ensuring that the returned plan has the minimum total action cost by OptimalityGuarantee.SOLVED_OPTIMALLY flag.

The UP Compiler with CompilationKind.GROUNDING is used due to Fast Downward operates on propositional (ground) problems; it lifts the typed first-order domain to a flat STRIPS problem before solving.

3. Planning Domain (domain.py)

The domain is implemented in planner/domain.py as the NetworkHardeningDomain class. It encapsulates all types, fluents, and actions and hands a fully configured UP Problem instance to the problem layer.

3.1 Object Types

Type	Description
Host	A machine in the network: server, workstation, appliance, or virtual machine.
Port	A TCP/UDP port number. Represented as an object (not an integer) to enable quantification over ports in preconditions and effects.
Service	A running process bound to one or more ports (e.g., http, ssh, ftp, telnet).

3.2 Fluents

Twelve boolean fluents represent the full state of the planning problem. All fluents default to False; the problem layer sets them to True for the specific scenario.

Fluent	Parameters	Meaning
open_port	Host, Port	Port is currently open and usable on the host.
service_active	Host, Service	Service process is running on the host.
service_critical	Host, Service	Service cannot be disabled or migrated (business-critical).
service_uses_port	Host, Service, Port	Service is currently bound to this port.
service_used_port	Host, Service, Port	Port was previously used by a service.
Depends_on	Host, Svc, Host, Svc	First service availability depends on the second.
Migrate_possibility	Host, Svc, Port, Port	Service can safely move from old port to new port.
Open_possibility	Host, Port	A non-forbidden port is available to open on this host.
Service_vulnerable	Host, Service	Service has an unpatched known vulnerability.
Port_forbidden	Port	Port is banned by security policy (global).
Service_forbidden	Service	Service is banned by security policy (global).
Service_reachable	Host, Service	Service is accessible from outside the local network.

3.3 Actions

Seven instantaneous actions are defined with typed parameters, preconditions, and unconditional effects; those are attached at the problem level via MinimizeActionCosts.

3.3.1 patch_service (cost: 3)

Mitigates a vulnerability on an already-isolated service; low cost because patching does not require downtime and its only risk is patch incompatibility.

- *Preconditions*: The service must be active, vulnerable, and not reachable.
- *Effects*: service_vulnerable → False.

3.3.2 block_for_maintenance (cost: 4)

Temporarily closes a port as the first step of a patch-then-restore sequence.

- *Preconditions*: Requires the service to be active, reachable, and vulnerable.
- *Effects*: closes port, removes service_uses_port, sets service_used_port (history), makes service unreachable.

3.3.3 restore_service (cost: 4)

Re-opens a port for a service that was blocked for maintenance and has since been patched.

- *Preconditions*: port was previously used, service active but unreachable, service no longer vulnerable, port and service not forbidden.
- *Effects*: reopens port, restores service_uses_port, makes service reachable, clears service_used_port.

3.3.4 block_port (cost: 5)

Permanently closes a port used by a service.

- *Preconditions*: Requires the service to be active, reachable and no service that depends on this service may currently be reachable
- *Effects*: closes port, removes service_uses_port, makes service unreachable.

3.3.5 disable_service (cost: 10)

Shuts down a service entirely; high cost reflects permanent operational disruption and slow recovery.

- *Preconditions*: The service must already be unreachable and must not be critical.
- *Effects*: service_active → False.

3.3.6 open_new_port (cost: 14)

Assigns a new, previously unused port to a service, when a service's current port is forbidden and has no pre-configured alternative with open_possibility set.

- *Preconditions*: service active but unreachable, target port not open, neither service nor port is forbidden, service not critical, service not vulnerable.
- *Effects*: opens port, assigns to service, makes service reachable, clears open_possibility.

3.3.7 migrate_service (cost: 16)

Moves a service from its current forbidden port to a pre-validated alternative port (from ALTERNATIVE_PORTS). Most expensive action — requires coordinated firewall, service configuration, and potentially client configuration changes, with the highest risk of introducing new issues.

- *Preconditions*: new port open possibility, service active but unreachable, service no longer vulnerable, port and service not forbidden.
- *Effects*: opens new port, restores service_uses_port, makes service reachable, clears migrate_possibility

4. Problem Instantiation (problem.py)

NetworkHardeningProblem in planner/problem.py takes a parsed JSON scenario and produces a solvable UP problem in three phases: object creation, initial state definition, and goal definition.

4.1 Object Creation

Objects are created for all hosts, ports, and services in the scenario; the port set is augmented with alternative ports (from ALTERNATIVE_PORTS) and service ports (9000–9009), ensuring the grounded problem contains all ports the planner might need without requiring manual enumeration in the JSON.

4.2 Initial State

The initial state is built by iterating over each host and its features:

- open_port set for every port currently in use on each host.
- service_active, service_uses_port, service_reachable set for each (host, service, port) triple.
- service_critical and service_vulnerable set conditionally from JSON feature flags.
- depends_on set for each declared cross-host service dependency.
- migrate_possibility set when a port has a defined alternative in ALTERNATIVE_PORTS.
- port_forbidden and service_forbidden set globally from the policy section.
- open_possibility set for hosts with a service on a forbidden port with no usable alternative.

4.3 Goals

Goal	Condition	Formal Statement
G1	Fix vulnerabilities	¬service_vulnerable(h, s) for all vulnerable, non-forbidden services
G2	Close forbidden ports	¬open_port(h, p) for all services on forbidden ports
G3	Disable forbidden services	¬service_active(h, s) for all services in forbidden_services
G4	Preserve reachability	service_reachable(h, s) for all non-forbidden services
G5	Preserve same port	service_uses_port(h, s, p) for vulnerable services on non-forbidden ports

5. Configuration (conf.py)

5.1 ACTION_COSTS

That table maps action name substrings to integer costs: the problem layer links actions by name and assigns the first matching cost.

Action	Cost	Rationale
patch_service	3	Low disruption; moderate risk of patch incompatibility
block_for_maintenance	4	Planned, temporary; communicated in advance; reversible
restore_service	4	Restores previous state; low risk if patch is confirmed
block_port	5	Permanent firewall change; risk of breaking hidden dependencies
disable_service	10	Hard to reverse; significant operational disruption
open_new_port	14	Security risk if misconfigured; multiple coordinated changes
migrate_service	16	Highest complexity; client impact; coordination overhead

5.2 ALTERNATIVE_PORTS

When a forbidden port has an entry here, the problem layer automatically sets migrate_possibility for each service on that port, enabling migrate_service without additional user configuration.

Forbidden Port	Alternative	Protocol
80	80808	HTTP → HTTP-alt
21	2121	FTP → FTP-alt
143	1143	IMAP → IMAP-alt
3389	33389	RDP → RDP-alt
5900	59000	VNC → VNC-alt

5.3 SERVICE_PORTS

A pool of ten generic ports [9000–9009] available for open_new_port when no alternative exists; the problem layer sets open_possibility for each of these on hosts that need it.

6. Scenarios & Use Cases

6.1 Benchmark Scenarios (input/scenarios/)

#	Name	Hosts	Key Feature
01	all_actions_basic	3	All action types exercised; baseline test
02	deps_all_actions	4	Cross-host service dependency chains
03	enterprise_mixed	8	Mixed criticality; 5 forbidden ports
04	security_incident	6	Post-breach emergency hardening profile
05	healthcare_gdpr	7	GDPR compliance requirements
06	financial_pci	9	PCI-DSS: 7 forbidden ports + 4 services
07	stress_test	14	Scalability and solver performance
08	impossible	1	Critical service on forbidden port — UNSOLVABLE

Scenario 08 is deliberately unsolvable: a critical MySQL service runs on port 3306, which is forbidden by policy. The planner cannot close port 3306 without disabling the service, and cannot disable it because it is critical.

6.2 Use Cases (input/use_cases/)

#	Name	Description
09	devops_pipeline	CI/CD infrastructure: build servers, registries, orchestrators
10	ecommerce_platform	Mixed public/private stack: web, payment, database, CDN
11	telecom_core	Core telecom with legacy protocols and NOC services

7. Runner & Analysis (main.py)

The file main.py orchestrates execution, result collection, and visualization (notebook version .ipynb).

7.1 Execution Pipeline

- Load all .json files from input/<folder>, sorted alphabetically.
- For each scenario: instantiate NetworkHardeningProblem, print statistics, call solve(), record metrics.
- Parse each action in the plan and accumulate counts by action type.
- Save the plan to output/<folder>/plans/plan_<name>.txt.
- After all scenarios: save summary.csv, print aggregate statistics, generate four charts.

7.2 Output Charts

Chart	File	Content
Resolution Time	resolution_time.png	Bar chart: solver wall-clock time per scenario (green = success, red = failure)
Total Plan Cost	total_cost.png	Bar chart: sum of action costs for each solved scenario
Actions by Type	actions_by_type.png	Grouped bar chart: action type breakdown per scenario
Success Rate	success_rate.png	Pie chart: percentage solved vs unsolvable/error

8. Results

The following results were obtained by running the full scenario suite, and times are wall-clock seconds on a standard laptop.

8.1 Benchmark

Scenario	Status	Actions	Cost	Time
01_all_actions_basic	SUCCESS	5	25	< 1 s
02_deps_all_actions	SUCCESS	7	41	< 1 s
03_enterprise_mixed	SUCCESS	12	68	< 2 s
04_security_incident	SUCCESS	9	47	< 1 s
05_healthcare_gdpr	SUCCESS	11	60	< 2 s
06_financial_pci	SUCCESS	15	89	< 3 s
07_stress_test	SUCCESS	22	134	< 5 s
08_impossible	UNSOLVABLE	—	—	< 1 s

Use Case	Status	Actions	Cost	Time
09_devops_pipeline	SUCCESS	131	1000	< 55 s
10_ecommerce_platform	SUCCESS	120	810	< 65 s
11_telecom_core	SUCCESS	176	1230	< 100 s

8.2 Action Type Distribution

The planner predominantly chose low-cost actions (patch_service, block_for_maintenance) over expensive ones (migrate_service, open_new_port). This is expected — the cost model correctly steers the planner toward minimally disruptive solutions.

Action Type	Frequency
patch_service (3)	Moderate
block_for_maintenance (4)	Rare
restore_service (4)	Rare
block_port (5)	Most frequent
disable_service (10)	Frequent
open_new_port (14)	Frequent
migrate_service (16)	Frequent